DECK 4 OF 12 · FRC JAVA CRASH COURSE

Drive the KitBot with no robot

The WPILib simulator, a little physics, and AdvantageScope

WPILib 2026 · Java · Command-based

OVERVIEW

What this deck covers

- What a simulator can and can’t show.
- A small physics model added to the Deck 2 drivetrain.
- **Driving the KitBot around a 2D field** with a controller or keyboard.
- What the model calculates every 20 milliseconds.

WHY SIMULATE

What a simulator gives you, and what it leaves out

What you get

No robot: test code at home on a laptop.

No risk: a wrong sign can’t hit anything.

No waiting for the mechanical team to finish.

Rough physics: close enough to learn from.

What it leaves out

Carpet: the wheels never slip.

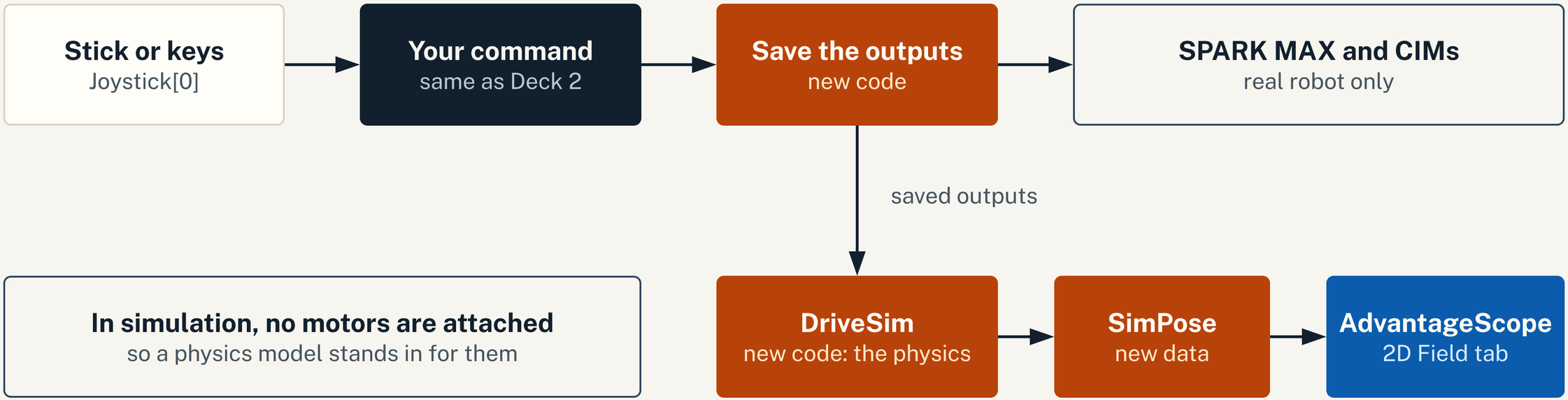
Wear: gearboxes and belts stay perfect.

Battery sag: our simulated battery stays at 12 V.

Walls: the robot drives straight through them.

THE PLAN

One path from your stick to the field



Schematic. A **pose** is where the robot is and which way it faces.

STEP 1: THE MODEL

A physics model of the KitBot

DriveSim.java

```
  private final DifferentialDrivetrainSim sim =
      DifferentialDrivetrainSim.createKitbotSim(
1         KitbotMotor.kDualCIMPerSide, KitbotGearing.k8p45,
1         KitbotWheelSize.kSixInch, null);

2 private final StructPublisher<Pose2d> posePublisher =
2     NetworkTableInstance.getDefault()
2         .getStructTopic("SimPose", Pose2d.struct).publish();

  public void step(double leftVolts, double rightVolts) {
3   sim.setInputs(leftVolts, rightVolts);
3   sim.update(LOOP_SECONDS);
4   posePublisher.set(sim.getPose());
  }
```

1. **Match the real KitBot**
   2 CIMs per side, 8.45:1 gearing, 6 inch wheels.

2. **Publish the pose**
   NetworkTables carries it to AdvantageScope.

3. **Step time forward**
   Give it volts, then move ahead 20 ms.

4. **Share the result**
   Each step sends the new pose.

STEP 2: THE OUTPUTS

Save each output on its way to the motors

CANDriveSubsystem.java

```
  // The last outputs we commanded, saved for the simulator.
1 private double leftOutput = 0.0;
1 private double rightOutput = 0.0;

  private final DifferentialDrive drive =
2     new DifferentialDrive(this::setLeft, this::setRight);

  private void setLeft(double output) {
    leftOutput = output;
3   leftLeader.set(output);
  }

  private void setRight(double output) {
    rightOutput = output;
3   rightLeader.set(output);
  }
```

1. **Our own copy**
   A SPARK's applied output needs extra REV code to update in sim.

2. **Methods, not motors**
   DifferentialDrive calls these with each output.

3. **The robot is unchanged**
   The real motors get the same set() call.

2027: set() becomes setThrottle()

STEP 3: THE SIM LOOP

Step the model in simulationPeriodic()

CANDriveSubsystem.java

```
  // Built only in simulation. On a real robot this stays null.
1 private final DriveSim sim =
1     RobotBase.isSimulation() ? new DriveSim() : null;

2 @Override
2 public void simulationPeriodic() {
3   double leftVolts = leftOutput * NOMINAL_VOLTS;
3   double rightVolts = rightOutput * NOMINAL_VOLTS;

    // Real motors stop when disabled. Saved outputs do not.
4   if (DriverStation.isDisabled()) {
      leftVolts = 0.0;
      rightVolts = 0.0;
    }
    sim.step(leftVolts, rightVolts);
  }
```

1. **Sim only**
   On the roboRIO this stays null.

2. **Called for you**
   The scheduler runs it every loop, only in sim.

3. **Output to volts**
   Deck 2 set 12 V compensation, so 0.5 means 6 V.

4. **Stop when disabled**
   Without this, the robot keeps driving after you disable it.

RUN IT

Start the simulator from VS Code

1. **Check Desktop Support is on**
   In build.gradle: includeDesktopSupport = true.

2. **Run WPILib: Simulate Robot Code**
   Press Ctrl+Shift+P and type it. Don’t use the run button by the WPILib logo.

3. **Keep Sim GUI selected and press Ok**
   The Simulation GUI window opens.

4. **Find the Robot State window**
   Its labels work like the Driver Station mode buttons.

WPILib simulation intro

docs.wpilib.org: Introduction to Robot Simulation

RUN IT

Put a driver on Joystick[0]

1. **Drag your device onto Joystick[0]**
   Drag from **System Joysticks** to **Joysticks**. No controller? Use **Keyboard 0**.

2. **Xbox controller: check Map gamepad**
   It keeps the right stick on axis 4, like the Driver Station.

3. **Keyboard: add a right stick**
   DS menu, **Keyboard 0 Settings**. Under **Axes**, set **Count** to 5.

4. **Keyboard: set the keys for Axis 4**
   Increase is D, **Decrease** is A. W and S already drive.

5. **Click Teleoperated in Robot State**
   Now push forward.

Sim GUI docs

docs.wpilib.org: Simulation User Interface

SEE IT

Connect AdvantageScope and add the field

1. **Open AdvantageScope**
   WPILib command palette: **Start Tool**, then **AdvantageScope**.

2. **Connect to the simulator**
   File > Connect to Simulator > Default, or Ctrl+Shift+K.

3. **Add a 2D Field tab**
   Use the plus button in the tab bar.

4. **Pick the 2026 field**
   Use the **Field** list in the panel under the view.

5. **Drag SimPose onto Poses**
   Find it in the sidebar. A robot appears near the blue end.

2D Field docs

docs.advantagescope.org: 2D Field

2027: field origin moves to the center of the field

COMMON MISTAKE

The robot ignores you

What you see	You press keys or move the stick, and the robot sits still.
Why it happens	Robot State isn't on **Teleoperated**, Joystick[0] is empty, or another window has keyboard focus.
The fix	Click **Teleoperated**. Drag your device onto Joystick[0]. Click the Sim GUI window before you use the keys.

COMMON MISTAKE

The robot drives the wrong way

What you see	You push forward and the robot backs up, spins, or crosses the screen the wrong way.
Why it happens	**Backs up:** a stick isn’t negated. **Spins:** one side has a flipped sign. **Wrong way on screen:** check the view first, because heading 0 faces the red wall.
The fix	Negate both sticks in RobotContainer. Pass both saved outputs to setInputs with no minus signs.

COMMON MISTAKE

AdvantageScope shows nothing

What you see	The simulator is running, but the field is empty or the title bar says “Searching”.
Why it happens	“Searching” means it hasn’t found the simulator. An empty field means the tab hasn’t been told what to draw.
The fix	Use Connect to Simulator, not Connect to Robot. Then drag SimPose onto **Poses**.

BEST PRACTICE

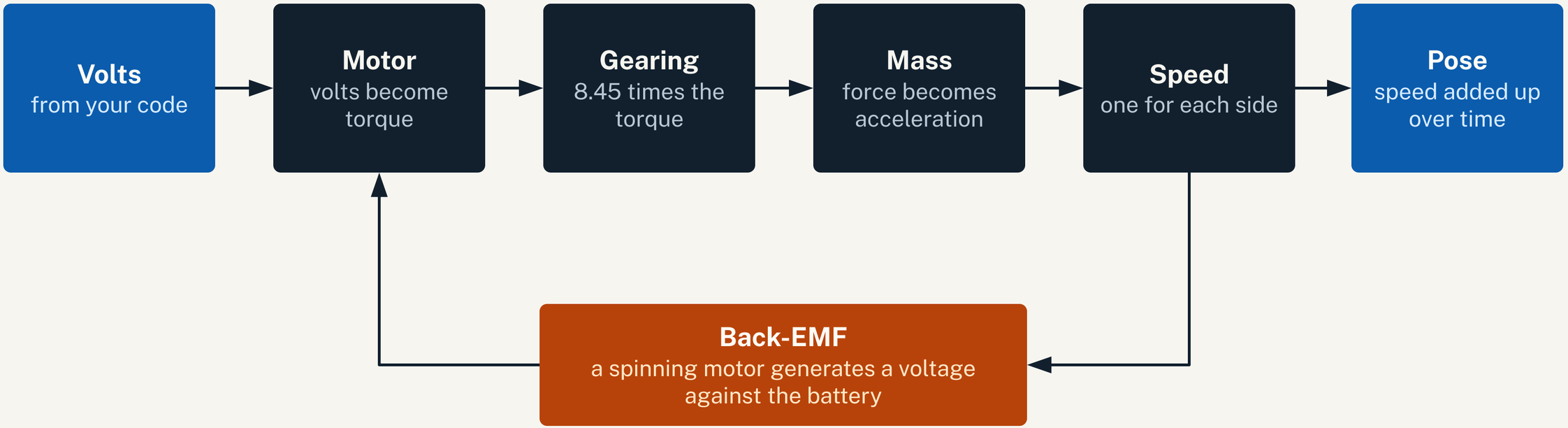
Keep sim code out of the real robot’s path

> **Keep sim code in simulationPeriodic() and DriveSim. Drive the same commands in sim and on the robot.**

Your commands never ask whether they are in a simulator, so what you test is what runs on the robot. Code that never runs on the roboRIO can’t break it at an event.

CONTROL THEORY

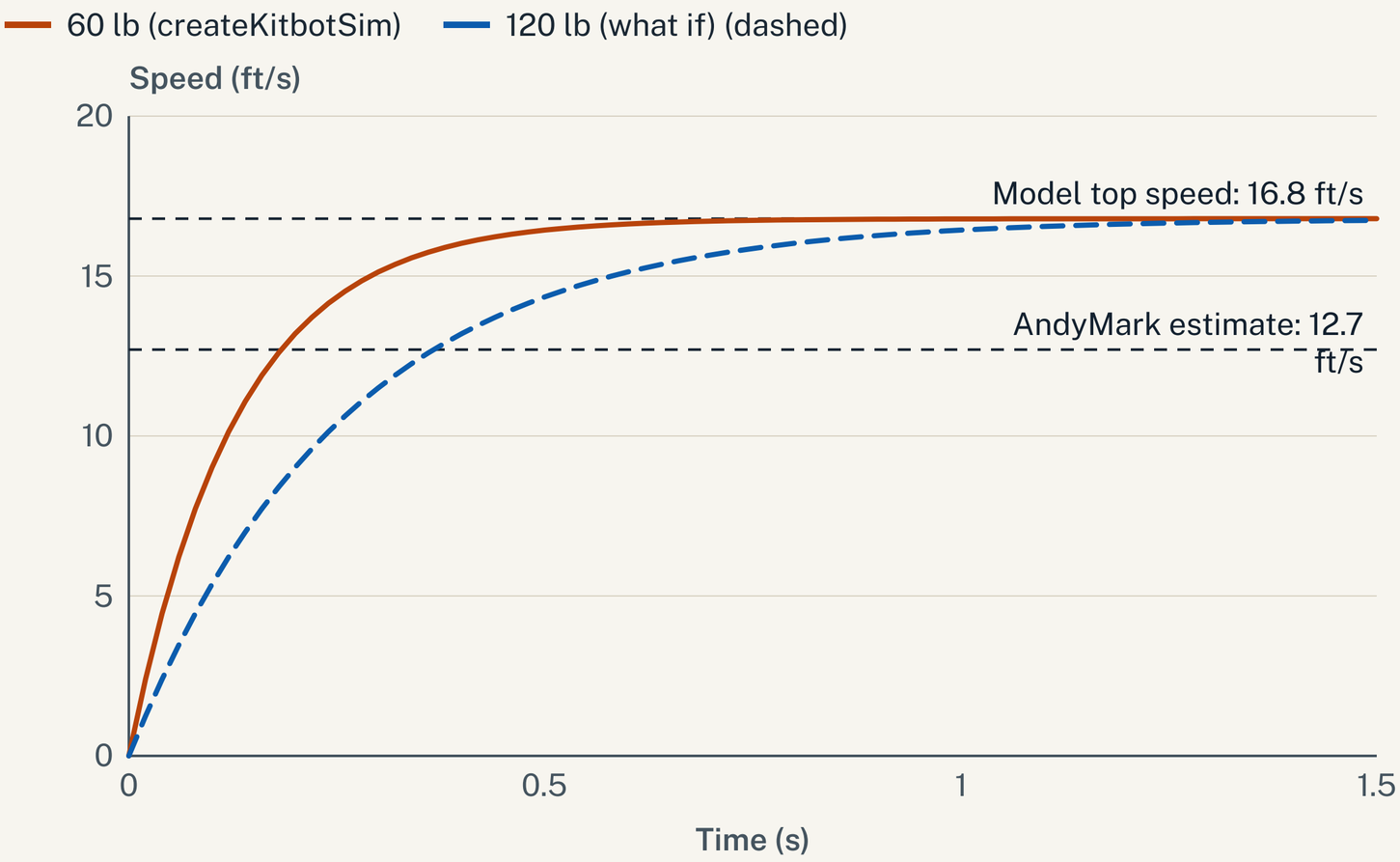
The plant model: what runs every 20 ms



Schematic. Not in this model: friction, tire slip, current limits and battery sag.

CONTROL THEORY

Full throttle from a standstill



1. **It flattens out**
   Back-EMF grows with speed, so torque falls.

2. **Mass changes the time**
   Double the mass, double the time. Same top speed.

3. **Why 16.8, not 12.7?**
   The 12.7 assumes CIMs loaded down to 4100 rpm. This model has no friction.

EXERCISE

Drive a figure-8, then change the robot

1. Drive a figure-8. Then time one full-stick trip across the field.

2. In DriveSim, change k8p45 to k5p95 or k12p75. Time the same trip.

3. Change it back. In RobotContainer, pass this::slewedForward to driveArcade.

4. Slam the stick from stop to full. Watch how the start changes.

Done when: you have two trip times.

NEXT SEASON

What changes for 2027

In 2026	Changing for 2027
edu.wpi.first packages	org.wpilib packages
Motor set()	Motor setThrottle()
AdvantageScope field origin at the blue wall corner	Default origin at the center of the field
Java 17 on the roboRIO	Java 25 on Systemcore

Alpha notes that may change. Everything in this deck is right for 2026.

RECAP

What this deck covered

- A simulator tests your logic and rough physics, not carpet, wear or battery sag.
- DriveSim turns saved outputs into a pose every 20 ms.
- simulationPeriodic() runs only in simulation, so the robot path stays clean.
- AdvantageScope draws SimPose on the 2D Field tab.

Next: Deck 5: Elastic, the dashboard for your drivers.

REFERENCE

Official docs



Simulation intro

docs.wpilib.org



Drivetrain sim tutorial

docs.wpilib.org



AdvantageScope live sources

docs.advantagescope.org



REVLib simulation

docs.revrobotics.com